用户标签分类大模型训练文档

项目概述

项目目标

基于用户对年画作品的交互行为（浏览、打分、评论、收藏等），训练一个大语言模型，为用户打上合适的年画风格和主体标签，实现个性化推荐。

技术栈

- **模型**: Qwen3-4B-Instruct-2507 (4B参数)
- **训练框架**: LLaMA-Factory
- **训练方法**: LoRA (Low-Rank Adaptation)
- **数据格式**: ShareGPT
- **环境**: Conda + Python 3.10+

数据处理流程

1. 原始数据结构

数据位于 `./data` 目录，包含以下CSV文件：

文件名	描述	记录数
年画作品.csv	年画作品信息（ID、名称、风格、主体）	作品总数
年画风格.csv	风格分类（ID、名称）	风格类别数
年画主体.csv	主体分类（ID、名称）	主体类别数
用户信息.csv	用户基本信息	用户总数
用户浏览年画物品记录.csv	浏览记录	浏览记录数
用户作品打分.csv	用户评分（1-5星）	打分记录数
用户评价.csv	文字评论	评论记录数
用户收藏记录.csv	收藏记录	收藏记录数

2. 数据处理策略

2.1 用户交互权重设计

根据用户行为的重要性，设置不同权重：

行为类型	权重	说明
收藏	5	最强烈的兴趣表达

行为类型	权重	说明
5星评分	2.5	高度喜欢
4星评分	2.0	比较喜欢
评论	2.0	有深度互动
浏览	1.0	基础兴趣

2.2 训练样本生成逻辑

筛选条件:

- 只选择正向交互（收藏、4星以上评分、有意义的评论）
- 每个用户至少2个正向交互才生成训练样本
- 每个用户最多取10个交互记录（避免数据倾斜）

样本构建:

1. 收集用户的具体交互行为及对应作品信息
2. 根据交互权重统计用户对不同风格和主体的偏好
3. 选取top-3风格和top-3主体作为用户标签
4. 构建system-user-assistant对话格式

2.3 训练样本格式示例

```
{
  "messages": [
    {
      "role": "system",
      "content": "你是一个专业的用户标签分类助手。你的任务是根据用户对年画作品的交互行为（浏览、打分、评论、收藏等），分析用户的偏好特征，为用户打上合适的年画风格和主体标签。"
    },
    {
      "role": "user",
      "content": "请根据以下用户的交互行为，为该用户分配合适的年画标签：\n\n1. 收藏了《春牛图》(杨柳青风格, 吉祥物主体)\n2. 打分5星了《门神》(杨柳青风格, 神像主体)\n3. 评论了《年年有余》(杨柳青风格, 吉祥物主体): \"色彩鲜艳，寓意美好\"\n\n请分析该用户的偏好特征，并为其打上年画风格和主体标签。"
    },
    {
      "role": "assistant",
      "content": "基于该用户的交互行为分析，该用户应被打上以下标签：\n风格标签: 杨柳青; 主体标签: 吉祥物, 神像\n\n分析理由：该用户对上述风格和主体的年画作品表现出明确的偏好，通过收藏、高分评价和评论等正向交互行为，反映出对这些类型作品的强烈兴趣。"
    }
  ]
}
```

3. 数据处理脚本

脚本: `prepare_training_data.py`

执行命令:

```
python prepare_training_data.py
```

输出:

- 训练数据文件: `train_data.json`
- 生成样本数: 4,951个

环境搭建

1. Conda环境创建

```
# 创建虚拟环境
conda create -n llamafactory python=3.10 -y

# 激活环境
conda activate llamafactory
```

2. 安装LLaMA-Factory

```
# 克隆仓库（如果还没有）
cd /home/ubuntu/category-cls
git clone https://github.com/hiyouga/LLaMA-Factory.git

# 进入目录
cd LLaMA-Factory

# 安装依赖
pip install -e ".[torch,metrics]" -i
https://pypi.tuna.tsinghua.edu.cn/simple
```

关键依赖:

- transformers
- datasets
- accelerate
- peft (for LoRA)
- trl
- torch

模型选择与下载

1. 模型选择

模型: Qwen3-4B-Instruct-2507

- **参数量**: 40亿 (4B)
- **来源**: Qwen/Qwen3-4B-Instruct-2507 @ HuggingFace
- **发布日期**: 2025年07月
- **架构**: Qwen3 (Transformer-based)

选择理由:

- 中文能力强：Qwen系列专门优化了中文理解
- 适中规模：4B参数在单GPU上可训练，资源消耗合理
- 指令遵循：Instruct版本经过指令微调，适合任务导向场景
- 最新版本：2507版本包含最新优化

2. 模型下载

下载脚本: `download_qwen.py`

```
from huggingface_hub import snapshot_download
import os

print("开始下载 Qwen3-4B-Instruct-2507 模型...")
model_dir = snapshot_download(
    repo_id='Qwen/Qwen3-4B-Instruct-2507',
    cache_dir='./models',
    resume_download=True
)
print(f"模型下载完成，保存路径: {model_dir}")
```

执行命令:

```
python download_qwen.py
```

下载位置: `./models/`

训练配置

1. 配置文件

文件: `train_config.yaml`

```
### 模型配置
model_name_or_path: Qwen/Qwen3-4B-Instruct-2507
```

```yaml
### 训练方法
stage: sft                      # Supervised Fine-Tuning
do_train: true
finetuning_type: lora           # 使用LoRA进行参数高效微调
lora_target: all                # 对所有线性层应用LoRA

### 数据集配置
dataset: user_tagging
template: qwen                  # 使用Qwen模板
cutoff_len: 1024                # 最大序列长度
max_samples: 5000               # 最大样本数
overwrite_cache: true
preprocessing_num_workers: 16

### 输出配置
output_dir: ./output/qwen3_user_tagging_lora
logging_steps: 10               # 每10步记录一次
save_steps: 500                 # 每500步保存一次
plot_loss: true                 # 绘制损失曲线
overwrite_output_dir: true

### 训练超参数
per_device_train_batch_size: 2      # 每个设备的batch size
gradient_accumulation_steps: 4      # 梯度累积步数（有效batch size = 2*4 = 8）
learning_rate: 5.0e-5               # 学习率
num_train_epochs: 3.0               # 训练轮数
lr_scheduler_type: cosine           # 余弦学习率衰减
warmup_ratio: 0.1                   # 预热比例
bf16: true                          # 使用BF16混合精度训练
ddp_timeout: 180000000              # 分布式训练超时时间

### 验证配置
val_size: 0.1                       # 验证集比例（10%）
per_device_eval_batch_size: 1
eval_strategy: steps                # 按步数评估
eval_steps: 500                     # 每500步评估一次
```

2. 数据集配置

文件: `dataset_info.json`

```json
{
  "user_tagging": {
    "file_name": "train_data.json",
    "formatting": "sharegpt",
    "columns": {
      "messages": "messages"
    },
    "tags": {
      "role_tag": "role",
      "content_tag": "content",
```

```
        "user_tag": "user",
        "assistant_tag": "assistant",
        "system_tag": "system"
      }
    }
  }
```

注意: 需要将 `dataset_info.json` 和 `train_data.json` 复制到 `LLaMA-Factory/data/` 目录下。

训练执行

1. 启动训练

脚本: `start_training.sh`

```
#!/bin/bash
set -e

echo "========================================"
echo "开始训练用户标签分类大模型"
echo "========================================"

# 激活conda环境
source /home/ubuntu/miniconda3/etc/profile.d/conda.sh
conda activate llamafactory

# 复制数据集配置到LLaMA-Factory
cp ../dataset_info.json data/
cp ../train_data.json data/

# 开始训练
echo "训练参数:"
echo "  模型: Qwen3-4B-Instruct-2507"
echo "  方法: LoRA"
echo "  Epochs: 3"
echo "  数据集: user_tagging (4951个样本)"
echo "========================================"

llamafactory-cli train ../train_config.yaml

echo "========================================"
echo "训练完成! "
echo "模型保存在: ./output/qwen3_user_tagging_lora"
echo "========================================"
```

执行命令:

```
cd LLaMA-Factory
bash ../start_training.sh
```

2. LoRA参数统计

```
总参数: 4,038,983,168 (4.04B)
可训练参数: 16,515,072 (16.5M)
可训练比例: 0.41%
```

LoRA应用位置:

- down_proj
- v_proj (Value projection)
- k_proj (Key projection)
- q_proj (Query projection)
- gate_proj
- up_proj
- o_proj (Output projection)

3. 训练数据划分

- **训练集**: 4,455 样本 (90%)
- **验证集**: 496 样本 (10%)

训练进度与监控

1. 当前训练状态

训练配置:

- 总训练步数: 1,671 steps
- 当前进度: Step 950/1,671 (57%)
- 当前轮次: Epoch 1.8/3.0
- 训练速度: ~1.32-1.33 秒/步

预计完成时间:

- 剩余步数: ~720 steps
- 预计剩余时间: ~16分钟

2. 损失变化趋势

训练阶段	训练损失	验证损失	说明
初始 (Step 10)	3.7714	-	开始阶段
Epoch 0.5 (Step 278)	0.0002	-	快速收敛

训练阶段	训练损失	验证损失	说明
Epoch 0.9 (Step 500)	~0.0005	0.000463	第一次评估
Epoch 1.8 (Step 950)	~0.0	-	当前状态

观察:

- 损失从3.77快速下降至接近0
- 模型收敛良好，无明显过拟合
- 验证损失与训练损失接近，泛化能力良好

3. 学习率调度

- **类型**: Cosine 余弦衰减
- **初始学习率**: 5e-5
- **预热比例**: 10% (前167步)
- **当前学习率**: ~2.08e-5 (Step 950)

4. 梯度范数

- **初始**: ~6-7 (较大)
- **当前**: ~0.001-0.01 (收敛良好)

模型使用指南

1. 模型加载

训练完成后，模型将保存在 `./output/qwen3_user_tagging_lora/` 目录。

使用LLaMA-Factory加载:

```
llamafactory-cli chat \
  --model_name_or_path Qwen/Qwen3-4B-Instruct-2507 \
  --adapter_name_or_path ./output/qwen3_user_tagging_lora \
  --template qwen
```

使用Python代码加载:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel

# 加载基础模型
base_model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen3-4B-Instruct-2507",
    device_map="auto",
    torch_dtype="auto"
)
```

```
# 加载LoRA适配器
model = PeftModel.from_pretrained(
    base_model,
    "./output/qwen3_user_tagging_lora"
)

# 加载分词器
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen3-4B-Instruct-2507")
```

2. 推理示例

```
def tag_user(interactions):
    """
    为用户打标签

    Args:
        interactions: 用户交互行为列表，格式如:
          ["收藏了《春牛图》(杨柳青风格, 吉祥物主体)",
           "打分5星了《门神》(杨柳青风格, 神像主体)"]

    Returns:
        用户标签
    """
    interaction_text = "\n".join(
        f"{i+1}. {item}" for i, item in enumerate(interactions)
    )

    messages = [
        {
            "role": "system",
            "content": "你是一个专业的用户标签分类助手。你的任务是根据用户对年画作品的交互行为（浏览、打分、评论、收藏等），分析用户的偏好特征，为用户打上合适的年画风格和主体标签。"
        },
        {
            "role": "user",
            "content": f"请根据以下用户的交互行为，为该用户分配合适的年画标签:\n\n{interaction_text}\n\n请分析该用户的偏好特征，并为其打上年画风格和主体标签。"
        }
    ]

    # 使用tokenizer构建输入
    text = tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True
    )

    inputs = tokenizer([text], return_tensors="pt").to(model.device)
```

```
    # 生成
    outputs = model.generate(
        **inputs,
        max_new_tokens=256,
        temperature=0.7,
        top_p=0.9,
        do_sample=True
    )

    response = tokenizer.decode(outputs[0][len(inputs.input_ids[0]):],
skip_special_tokens=True)
    return response

# 使用示例
user_interactions = [
    "收藏了《春牛图》(杨柳青风格, 吉祥物主体)",
    "打分5星了《门神》(杨柳青风格, 神像主体)",
    "评论了《年年有余》(杨柳青风格, 吉祥物主体): \"色彩鲜艳, 寓意美好\""
]

tags = tag_user(user_interactions)
print(tags)
```

3. 批量推理

```
def batch_tag_users(user_interaction_list, batch_size=8):
    """批量为用户打标签"""
    results = []

    for i in range(0, len(user_interaction_list), batch_size):
        batch = user_interaction_list[i:i+batch_size]
        batch_results = [tag_user(interactions) for interactions in batch]
        results.extend(batch_results)

    return results
```

4. 合并LoRA到基础模型（可选）

如果需要部署，可以将LoRA权重合并到基础模型：

```
from peft import PeftModel
from transformers import AutoModelForCausalLM

# 加载并合并
base_model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen3-4B-Instruct-
2507")
model = PeftModel.from_pretrained(base_model,
"./output/qwen3_user_tagging_lora")
merged_model = model.merge_and_unload()
```

```
# 保存合并后的模型
merged_model.save_pretrained("./output/qwen3_user_tagging_merged")
tokenizer.save_pretrained("./output/qwen3_user_tagging_merged")
```